Trabajo Práctico · Unidad 1 · Semana 2

Laboratorio de concurrencia e IA como motor primario · Base de Datos II · Tecnicatura Universitaria en Programación · UTN

> **Alumno:** Moran, Joaquín Leandro · Trabajo individual.
> **Repositorio:** github.com/moranjoaquin487-lgtm/tpi-food-store, carpeta `trabajos-practicos/unidad1/semana2_laboratorio_concurrencia`.
> **Proyecto:** Food Store, con el `schema.sql` del TP1, sobre PostgreSQL 18.6 en mi PC (Windows, Git Bash). **IA:** OpenCode, Kiro y Claude; qué hizo cada una está en las DUIA de cada parte.

Entregable	En el repositorio	En este documento
Parte 0 · Protocolo de seguridad	`protocolo_seguridad.md`, `AGENTS.md`, `.kiro/steering/`	Parte 0
Parte 1 · Script de restricciones + DUIA	`specs/spec_restricciones.md`, `food-store/restricciones.sql`, `docs/duia/duia_parte1.md`	Parte 1 y anexo A
Parte 2 · `informe_concurrencia.md` + DUIA	`docs/informe_concurrencia.md`, `docs/explicaciones_ia/`, `docs/duia/duia_parte2.md`	Parte 2 y anexo B
Parte 3 · `ejercicio_lectura_critica.md` + DUIA	`docs/ejercicio_lectura_critica.md`, `docs/duia/duia_parte3.md`	Parte 3
Defensa oral	Presencial; no forma parte del repositorio	

Historial de commits (`git log` de la carpeta, del primero al último)

Commit	Mensaje
`b3c82ba`	Semana 2: base propia del Food Store con el esquema del TP1, datos iniciales y copia de trabajo
`0b94321`	Parte 0: protocolo de seguridad adaptado a mi entorno
`353c6f9`	Parte 0: AGENTS.md generado con /init de OpenCode, con las reglas del protocolo
`7370aa8`	Parte 0: steering docs de Kiro con las convenciones del esquema
`571c9cc`	Parte 1: spec de las reglas A, B y C, antes de generar el código
`b4343d2`	Parte 1: el motor garantiza el precio congelado al vender, que no se dé de baja una categoría con productos activos y que un pedido no tenga fecha futura
`e846d21`	Parte 2: cuatro escenarios reproducidos con dos sesiones concurrentes
`3871360`	Parte 2: explicaciones de la IA de los cuatro escenarios, copiadas tal cual
`60d3359`	Parte 2: verificación en el motor de lo que propuso la IA
`c7cc4c2`	Parte 3: efecto real y corrección de los dos scripts de lectura crítica
`1f5f6f7`	Parte 2: guarda las respuestas de la IA de los escenarios 2 a 4, que habían quedado sin pegar
`e0a13db`	Semana 2: informe de concurrencia, DUIA de las tres partes y README actualizado
`5c3df14`	Semana 2: saca las fechas de las explicaciones de la IA y de la DUIA

Cada parte tiene sus propios commits, en el orden que pide la consigna: el protocolo antes que la Parte 1, y la spec antes que el código generado.

Parte 0 · Protocolo de seguridad (`protocolo_seguridad.md`)

Se aplica a **todo** script que escriba en la base, propio o generado por IA, sin excepción.

Mi entorno

Motor	PostgreSQL 18.6, local
Sistema	Windows, con Git Bash como terminal (`psql`, `createdb`, `pg_dump`, `pg_restore`)
Usuario de la base	`postgres`
Contraseña	Se carga una vez por sesión de Git Bash con `read -s -p "Contraseña de postgres: " PGPASSWORD; echo; export PGPASSWORD`. Nunca se escribe en un archivo del repo.
Agentes de IA	OpenCode y Kiro, sobre esta carpeta

Bases

Base	Rol
`foodstore_plantilla`	Esquema y datos iniciales (`food-store/schema.sql` y `data.sql`). **No se modifica.**
`foodstore_trabajo`	Copia de trabajo. Es la única base sobre la que se ejecutan scripts.

Paso 1 — Copia

Todo se ejecuta sobre la copia, nunca sobre la plantilla:

```
createdb -T foodstore_plantilla foodstore_trabajo
```

Si la copia quedó inservible, se descarta y se rehace (`scripts/crear_base.sh` recrea las dos bases desde los archivos del repo):

```
dropdb foodstore_trabajo
createdb -T foodstore_plantilla foodstore_trabajo
```

Antes de correr cualquier script, confirmar contra qué base apunta la sesión:

```
SELECT current_database();   -- tiene que decir foodstore_trabajo
```

Paso 2 — Transacción

Todo script que escribe corre primero dentro de una transacción que termina en `ROLLBACK`, para leer el efecto real (filas afectadas, mensajes, resultado de los controles) antes de confirmar nada:

```
BEGIN;
-- script a evaluar
-- controles: SELECT de verificación, casos válidos e inválidos
ROLLBACK;   -- recién después de revisar la salida, se repite terminando en COMMIT
```

- En `psql`, `\set ON_ERROR_ROLLBACK on` hace que un caso que falla a propósito no aborte toda la prueba: `psql` pone un `SAVEPOINT` antes de cada sentencia.
- `EXPLAIN ANALYZE` sobre `INSERT`, `UPDATE` o `DELETE` ejecuta la sentencia de verdad: también va dentro de `BEGIN ... ROLLBACK`.

Paso 3 — Respaldo

Antes de cualquier cambio estructural (`CREATE TRIGGER`, `ALTER`, `DROP`, migraciones), se respalda la copia de trabajo:

```
pg_dump -F c -f "backups/foodstore_trabajo_$(date +%Y%m%d_%H%M).dump" foodstore_trabajo
```

- Los respaldos viven en `backups/`, que está en `.gitignore`: son binarios y se regeneran.
- Para comprobar que el respaldo sirve: `pg_restore -l backups/<archivo>.dump` lista su contenido.
- Para volver atrás:

```
pg_restore -d foodstore_trabajo --clean --if-exists "backups/<archivo>.dump"
```

`scripts/aplicar_restricciones.sh --confirmar` hace el respaldo automáticamente antes de aplicar, y si el respaldo falla no aplica nada.

Con agentes de IA

- OpenCode se usa en modo **Plan** primero: describe qué va a hacer sin tocar archivos. Recién después de revisar el plan se pasa a modo Build.
- Nada generado se aplica sin leer el diff completo (`git diff`) línea por línea.
- Los agentes no ejecutan comandos contra la base: los scripts los corro yo, siguiendo los tres pasos de arriba.
- No se le pasan credenciales a ningún agente. OpenCode se abre en una ventana de Git Bash distinta de la que tiene `PGPASSWORD` cargada: si intentara conectarse a la base, no podría.

Además, `AGENTS.md` se generó con `/init` de OpenCode y los steering docs de Kiro quedaron en `.kiro/steering/` (detalle en la DUIA de la Parte 1).

Parte 1 · Integridad versionada con OpenCode

Spec (`specs/spec_restricciones.md`, commiteada antes de generar el código)

Parte 1 de la Semana 2. Esquema: `food-store/schema.sql`. Motor: PostgreSQL 18.

Son tres reglas que hoy dependen de que la aplicación se acuerde de validarlas. La spec se escribió antes de generar el código con OpenCode; la redacté con ayuda de Claude (ver la DUIA de la Parte 1).

Regla A — Precio congelado al vender

Al insertar una fila en `detalle_pedido`, `precio_unitario` tiene que ser igual a `producto.precio` del producto vendido en ese momento. Una vez registrada la fila, `precio_unitario` no se puede modificar.

Regla B — Baja de una categoría

`categoria.activo` no puede pasar de `TRUE` a `FALSE` mientras la categoría tenga al menos un producto con `producto.activo = TRUE`.

Regla C — Fecha de un pedido

`pedido.fecha` no puede ser posterior al momento en que se inserta o se modifica la fila.

Casos de prueba

#	Regla	Caso	Resultado esperado
1	A	Agregar la Napolitana al pedido de Jorge Díaz con `precio_unitario` igual a su precio actual	Se inserta
2	A	Agregar la Cerveza 1L al pedido de Jorge Díaz con un `precio_unitario` distinto del actual	Error
3	A	Cambiar el `precio_unitario` de una línea ya registrada	Error
4	A	Cambiar `producto.precio` de la Muzzarella, que ya tiene ventas	Se acepta, y las líneas vendidas conservan su precio
5	B	Dar de baja la categoría Promociones, que no tiene productos	Se acepta
6	B	Dar de baja la categoría Pizzas, que tiene productos activos	Error
7	C	Registrar un pedido con la fecha de este momento	Se inserta
8	C	Registrar un pedido con fecha de mañana	Error

Fuera de alcance

- Los datos cargados antes de crear las reglas no se revisan: tienen precios históricos, como la Muzzarella a 1000,00 en el primer pedido (R4).
- Dar de alta un producto activo en una categoría que ya está dada de baja: la Regla B controla solo la baja de la categoría.
- El stock no forma parte de estas reglas.

El script generado, `food-store/restricciones.sql`, está completo en el anexo A.

DUIA de la Parte 0 y la Parte 1 (`docs/duia/duia_parte1.md`)

Declaración de uso de IA de la puesta a punto (Parte 0) y de las restricciones de integridad (Parte 1).

OpenCode — restricciones de las reglas A, B y C

Campo	Completar
Herramienta	OpenCode 1.19.5, con el modelo Big Pickle que trae incluido (no configuré otro proveedor con `/connect`). Lo usé desde PowerShell, porque en Git Bash la interfaz no se dibujaba bien, y en una ventana sin la contraseña de la base.
Spec o prompt utilizado	La spec es `specs/spec_restricciones.md`, commiteada antes de generar el código (commit `571c9cc`). El pedido del plan, en modo Plan, es el texto exacto de la sección 1 de `prompts/parte1_opencode.md`. El pedido de escritura, en modo Build, fue: «El plan está bien. Para la Regla C dejá el trigger, no el CHECK con now(). Escribí los dos archivos.»

Campo	Completar
Qué generó	El plan está guardado con `/export` en `docs/plan_opencode_parte1.md`. Generó `food-store/restricciones.sql` (114 líneas): tres funciones PL/pgSQL con sus triggers `BEFORE ... FOR EACH ROW`, una por regla, con `DROP TRIGGER IF EXISTS` y `CREATE OR REPLACE FUNCTION` para poder correrlo más de una vez, y mensajes de error en español que nombran la regla. También generó `food-store/pruebas_restricciones.sql` (94 líneas): los 8 casos de la spec en orden, buscando las filas por nombre o email, y el SELECT del caso 4.
Qué se aceptó	Los dos archivos quedaron tal cual los escribió OpenCode. El propio OpenCode retocó el comentario del encabezado para aclarar que los `BEGIN ... END` de las funciones son sintaxis de PL/pgSQL y no control de transacción.
Qué se modificó o descartó, y por qué	No corregí ninguna línea a mano. En el plan, OpenCode preguntó si la Regla C iba con trigger o con `CHECK (fecha <= now())`. Elegí el trigger por dos motivos: un CHECK tiene que ser inmutable y `now()` no lo es, y el trigger permite un mensaje que nombra la regla. Antes de aprobar, revisé que los productos que elegía para los casos 1 a 3 no chocaran con la clave primaria de `detalle_pedido`.
Verificación realizada	Leí el diff completo con `git diff --staged` y controlé con `grep` que no hubiera `BEGIN`, `COMMIT` ni `ROLLBACK` sueltos (no imprimió nada). Corrí `bash scripts/aplicar_restricciones.sh`, que aplica y prueba dentro de `BEGIN ... ROLLBACK`. La primera vez falló la conexión porque escribí mal la contraseña; la segunda corrida (`parte1_1_prueba_con_rollback.txt`) dio lo que pide la spec en los 8 casos: pasaron el 1, 4, 5 y 7 (`INSERT 0 1` o `UPDATE 1`), y el 2, 3, 6 y 8 dieron `ERROR` con el mensaje de su regla. Después corrí `--confirmar`: hizo el respaldo en `backups/`, aplicó con `COMMIT` (`parte1_2_aplicado.txt`) y repitió los casos (`parte1_3_pruebas.txt`). En el caso 4, las tres ventas de la Muzzarella conservaron 1000.00, 1050.00 y 1050.00 con el precio de lista en 1100.00.

OpenCode — `AGENTS.md` (Parte 0)

Campo	Completar
Herramienta	OpenCode 1.19.5, modelo Big Pickle.
Spec o prompt utilizado	El comando `/init` y el pedido «Reglas del protocolo», texto exacto en `prompts/parte0_agentes.md`.
Qué generó	`AGENTS.md`: contexto del repo, reglas de base de datos, comandos, mapa del proyecto, convenciones del esquema y la sección «Reglas para agentes».
Qué se aceptó	El archivo completo, tal cual.
Qué se modificó o descartó, y por qué	La primera vez corrí `/init` en modo Plan y no escribió el archivo; lo repetí en modo Build. OpenCode preguntó dónde escribirlo y elegí la carpeta de la Semana 2, que es la que indican el README y los prompts.
Verificación realizada	Leí el archivo entero y comprobé que todo lo que menciona existe: los tipos y claves de `schema.sql`, las variables de los scripts de concurrencia, el respaldo con `pg_dump`, y en la raíz del repo `ejecutar_todo.sh`, la carpeta `sql/` y las reglas de LF de `.gitattributes`. No encontré nada inventado.

Kiro — steering docs (Parte 0)

Campo	Completar
Herramienta	Kiro, con el selector de modelo en Auto. Lo instalé para este TP.
Spec o prompt utilizado	Generación automática (panel de Kiro → *Agent Steering & Skills* → **+** → *Project steering files*), el pedido «Convenciones del esquema» y un pedido de corrección. Los tres están en `prompts/parte0_agentes.md`.
Qué generó	`.kiro/steering/product.md`, `tech.md`, `structure.md` y `esquema.md`.
Qué se aceptó	Los cuatro archivos, después de la corrección.
Qué se modificó o descartó, y por qué	Al compararlos con `schema.sql`, `data.sql` y el README encontré tres errores: `product.md` decía que la Parte 3 trataba de concurrencia (es la lectura crítica de dos scripts), `structure.md` decía que `data.sql` tiene 5 pedidos (son 7) y a `esquema.md` le faltaba la regla R3. Le pedí a Kiro que los corrigiera sin cambiar nada más.
Verificación realizada	Revisé los cuatro archivos contra el esquema, los datos y los scripts, y después de la corrección confirmé los tres cambios con `grep`.

Claude (Anthropic)

Campo	Completar
Herramienta	Claude (Anthropic).
Spec o prompt utilizado	Le pasé la consigna del TP y le pedí que me preparara el paquete de trabajo y me guiara paso a paso.
Qué generó	Con su ayuda se redactaron la spec de las reglas A, B y C, el protocolo de seguridad, `data.sql`, los scripts (`crear_base.sh`, `aplicar_restricciones.sh`), los pedidos para OpenCode y Kiro, y la guía de pasos. También señaló los errores de los steering docs y revisó conmigo el plan y el diff de OpenCode. Esta DUIA la redactó Claude con mis datos.
Qué se aceptó	La spec y los scripts se usaron como estaban.
Qué se modificó o descartó, y por qué	Las decisiones sobre lo que generaron OpenCode y Kiro, detalladas arriba.
Verificación realizada	Leí la spec y el protocolo antes de commitearlos, y corrí cada script en mi PC. Los resultados de las evidencias son los de esas corridas.

Lo que hice yo

Cloné el repo, creé las bases y controlé los conteos, leí el PDF del TP1, el protocolo, la spec, el plan y el diff, decidí qué aceptar de cada herramienta, corrí todos los scripts y commiteé cada paso. Las restricciones las tengo que poder explicar línea por línea en la defensa.

Parte 2 · Informe de concurrencia (`docs/informe_concurrencia.md`)

Parte 2 de la Semana 2. Base: `foodstore_trabajo`, copia de `foodstore_plantilla`, en PostgreSQL 18.6.

Los cuatro escenarios se corrieron con `bash scripts/laboratorio_concurrencia.sh reproducir`. El script abre dos conexiones de `psql` a la vez y la sesión B arranca 0,5 s después que la A. Los comandos de cada sesión están en `food-store/concurrencia/`, y la salida completa de cada escenario, en `evidencias/`. Después de cada escenario, el script deja los datos como estaban: restaura el precio y el stock, y borra el pedido de prueba.

Las explicaciones de la IA se pidieron a Kiro (modelo Auto) en un chat nuevo por escenario, pegando el pedido completo que genera el script (`prompts/explicacion_escenario_N.md`). Las respuestas están copiadas tal cual en `docs/explicaciones_ia/`. Para verificarlas, se repitió cada experimento con `bash scripts/laboratorio_concurrencia.sh verificar` y con `bash scripts/verificar_lock_table.sh`.

Resumen

#	Escenario	Qué se observó	Qué lo evita, según el motor	¿Acertó la IA?
1	Lectura no repetible	1050.00 → 1150.00	REPEATABLE READ y SERIALIZABLE	Sí
2	Lectura fantasma	2 → 3	REPEATABLE READ, SERIALIZABLE y `LOCK TABLE ... IN SHARE MODE`	Sí
3	Espera por bloqueo	B esperó 1,38 s y leyó stock 59	NOWAIT, SKIP LOCKED y `lock_timeout` evitan la espera	En parte: se equivocó sobre SERIALIZABLE
4	Interbloqueo	A abortada con `40P01`	Tomar las filas en el mismo orden	Sí, con dos imprecisiones

Escenario 1 — Lectura no repetible

Escenario. La sesión A lee dos veces el precio de la Muzzarella dentro de la misma transacción, en READ COMMITTED, mientras la sesión B lo sube y confirma.

Cómo se reprodujo.

Sesión A (`e1_A.sql`, con `:nivel = READ COMMITTED`):

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL :nivel;
SHOW transaction_isolation;
SELECT precio AS primera_lectura FROM producto WHERE nombre = 'Muzzarella';
SELECT pg_sleep(1.5);  -- mientras tanto, la sesión B cambia el precio y confirma
SELECT precio AS segunda_lectura FROM producto WHERE nombre = 'Muzzarella';
COMMIT;
```

Sesión B (`e1_B.sql`), sin `BEGIN`, así que se confirma sola:

```
UPDATE producto SET precio = precio + 100 WHERE nombre = 'Muzzarella';
```

Qué se observó (`e1_no_repetible_read_committed.txt`):

Sesión	Comando	Salida
A	`SHOW transaction_isolation`	`read committed`
A	primera lectura	`1050.00`
B	`UPDATE`	`UPDATE 1`
A	segunda lectura	`1150.00`
A	`COMMIT`	`COMMIT`

La misma consulta, dentro de la misma transacción, devolvió dos valores distintos.

Explicación de la IA (Kiro, modelo Auto; tal cual en `escenario_1.md`). Identifica una lectura no repetible: en READ COMMITTED cada `SELECT` ve lo último confirmado. Propone REPEATABLE READ o SERIALIZABLE, que mantienen el mismo snapshot durante toda la transacción.

Verificación en el motor. Se repitió con la sesión A en los dos niveles propuestos:

Nivel de A	Primera lectura	Segunda lectura	Evidencia
REPEATABLE READ	1050.00	1050.00	`e1_verificacion_repeatable_read.txt`
SERIALIZABLE	1050.00	1050.00	`e1_verificacion_serializable.txt`

En los dos casos, la sesión B igual confirmó su `UPDATE 1`: no quedó bloqueada, como dijo la IA.

Conclusión. La explicación se confirmó. REPEATABLE READ alcanza para evitar la lectura no repetible. Un matiz: Kiro dice que el snapshot se toma "al inicio de la transacción". En PostgreSQL se toma en la primera consulta de la transacción, no en el `BEGIN`. Acá no cambia el resultado, porque la primera lectura fue antes del `UPDATE` de B.

Escenario 2 — Lectura fantasma

Escenario. La sesión A cuenta dos veces los pedidos de Ana Gómez, en READ COMMITTED, mientras la sesión B registra un pedido nuevo de Ana.

Cómo se reprodujo.

Sesión A (`e2_A.sql`, con `:nivel = READ COMMITTED`):

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL :nivel;
SHOW transaction_isolation;
SELECT COUNT(*) AS primer_conteo FROM pedido p JOIN cliente c ON c.id = p.cliente_id WHERE c.email =
'ana.gomez@mail.com';
SELECT pg_sleep(1.5);  -- mientras tanto, la sesión B registra un pedido nuevo de Ana
SELECT COUNT(*) AS segundo_conteo FROM pedido p JOIN cliente c ON c.id = p.cliente_id WHERE c.email =
'ana.gomez@mail.com';
COMMIT;
```

Sesión B (`e2_B.sql`), sin `BEGIN`:

```
INSERT INTO pedido (forma_pago, cliente_id)
SELECT 'EFECTIVO', id FROM cliente WHERE email = 'ana.gomez@mail.com';
```

Qué se observó (`e2_fantasma_read_committed.txt`):

Sesión	Comando	Salida
A	`SHOW transaction_isolation`	`read committed`
A	primer conteo	`2`
B	`INSERT`	`INSERT 0 1`
A	segundo conteo	`3`
A	`COMMIT`	`COMMIT`

Apareció una fila nueva que cumple el mismo `WHERE`. El pedido nuevo tiene fecha `now()` por defecto, así que la Regla C de la Parte 1 lo acepta.

Explicación de la IA (Kiro, modelo Auto; tal cual en `escenario_2.md`). Identifica una lectura fantasma y propone REPEATABLE READ. Aclara que el estándar SQL no exige que ese nivel evite los fantasmas, pero que en PostgreSQL sí los evita. También menciona SERIALIZABLE y, como alternativa sin cambiar de nivel, `LOCK TABLE pedido IN SHARE MODE` al inicio de la transacción de A.

Verificación en el motor. Se probaron las tres propuestas. La de `LOCK TABLE` no estaba en el script `verificar`, así que se agregó `scripts/verificar_lock_table.sh`, que corre el mismo escenario con A en READ COMMITTED y el bloqueo de tabla.

Sesión A	Primer conteo	Segundo conteo	Sesión B	Evidencia
REPEATABLE READ	2	2	`INSERT 0 1`, sin esperar	`e2_verificacion_repeatable_read.txt`
SERIALIZABLE	2	2	`INSERT 0 1`, sin esperar	`e2_verificacion_serializable.txt`

Sesión A	Primer conteo	Segundo conteo	Sesión B	Evidencia
READ COMMITTED + `LOCK TABLE pedido IN SHARE MODE`	2	2	`INSERT 0 1`, después de esperar 916 ms	`e2_verificacion_lock_table.txt`

Conclusión. La explicación se confirmó en las tres propuestas. Hay una diferencia de costo que coincide con lo que dijo la IA: REPEATABLE READ y SERIALIZABLE evitan el fantasma sin hacer esperar a nadie, porque A sigue viendo su snapshot. `LOCK TABLE` lo evita haciendo esperar a B hasta que A confirma, y además bloquea cualquier INSERT, UPDATE o DELETE sobre toda la tabla `pedido` mientras dura la transacción. Para este caso, REPEATABLE READ es la mejor opción.

Escenario 3 — Espera por bloqueo

Escenario. Las dos sesiones piden `FOR UPDATE` sobre la misma fila (la Coca 1.5L). La sesión A la toma, espera 2 s, descuenta una unidad de stock y confirma.

Cómo se reprodujo.

Sesión A (`e3_A.sql`):

```
BEGIN;
SELECT id, stock FROM producto WHERE nombre = 'Coca 1.5L' FOR UPDATE;
SELECT pg_sleep(2);  -- mientras tanto, la sesión B pide la misma fila
UPDATE producto SET stock = stock - 1 WHERE nombre = 'Coca 1.5L';
COMMIT;
```

Sesión B (`e3_B.sql`, con `:modo` vacío):

```
BEGIN;
SELECT id, stock FROM producto WHERE nombre = 'Coca 1.5L' FOR UPDATE :modo;
SELECT round(extract(epoch FROM clock_timestamp() - now())::numeric, 2) AS segundos_desde_begin;
COMMIT;
```

Qué se observó (`e3_espera_por_bloqueo.txt`):

Sesión	Comando	Salida
A	`SELECT ... FOR UPDATE`	`id 9, stock 60`
B	`SELECT ... FOR UPDATE`	queda esperando
A	`UPDATE` y `COMMIT`	`UPDATE 1`, `COMMIT`
B	`SELECT ... FOR UPDATE`	`id 9, stock 59`
B	`segundos_desde_begin`	`1.38`

La sesión B esperó a que A liberara la fila y después leyó el valor ya confirmado.

Explicación de la IA (Kiro, modelo Auto; tal cual en `escenario_3.md`). Aclara que no es una anomalía sino el comportamiento buscado de `FOR UPDATE`. Para no esperar, propone `FOR UPDATE NOWAIT` o `FOR UPDATE SKIP LOCKED`. Además afirma que "el nivel de aislamiento no cambia esto" y que los bloqueos de fila "aplican en todos" los niveles.

Verificación en el motor.

Variante	Resultado en la sesión B	Evidencia
`FOR UPDATE NOWAIT`	`ERROR: 55P03: no se pudo bloquear un "lock" en la fila`, a los 0,01 s	`e3_verificacion_nowait.txt`
`FOR UPDATE SKIP LOCKED`	`(0 filas)`, a los 0,00 s	`e3_verificacion_skip_locked.txt`
`lock_timeout = '1s'` (no lo propuso la IA)	`ERROR: 55P03: ... se agotó el tiempo de espera de "locks"`, a los 1,03 s	`e3_verificacion_lock_timeout.txt`
Las dos sesiones en SERIALIZABLE	Espera 1,40 s y después `ERROR: 40001: no se pudo serializar el acceso debido a un update concurrente`	`e3_verificacion_serializable.txt`

Conclusión. La explicación se confirmó en lo central: NOWAIT falla enseguida y SKIP LOCKED saltea la fila. Pero el motor **desmiente** que el nivel de aislamiento no cambie nada. En SERIALIZABLE, B también espera, pero cuando A confirma no lee el 59: falla con `40001`, porque la fila cambió después de que empezó su transacción. Según la documentación de PostgreSQL, en REPEATABLE READ pasa lo mismo, pero eso no se probó en este laboratorio. Es una diferencia importante: en READ COMMITTED, B sigue con el dato nuevo; en los niveles más altos, la aplicación tiene que reintentar la transacción. Si lo que se busca es no esperar, sirven NOWAIT, SKIP LOCKED o `lock_timeout`.

Escenario 4 — Interbloqueo

Escenario. Las dos sesiones toman las mismas dos filas en orden cruzado. A toma la Napolitana y después pide la Fugazzeta; B toma la Fugazzeta y después pide la Napolitana.

Cómo se reprodujo.

Sesión A (`e4_A.sql`):

```
BEGIN;
UPDATE producto SET stock = stock WHERE nombre = 'Napolitana';  -- bloquea la fila sin cambiar el valor
SELECT pg_sleep(1);
UPDATE producto SET stock = stock WHERE nombre = 'Fugazzeta';
SELECT round(extract(epoch FROM clock_timestamp() - now())::numeric, 2) AS segundos_desde_begin;
COMMIT;
```

Sesión B (`e4_B.sql`, con `:primero` = `Fugazzeta` y `:segundo` = `Napolitana`):

```
BEGIN;
UPDATE producto SET stock = stock WHERE nombre = :'primero';
SELECT pg_sleep(1);
UPDATE producto SET stock = stock WHERE nombre = :'segundo';
SELECT round(extract(epoch FROM clock_timestamp() - now())::numeric, 2) AS segundos_desde_begin;
COMMIT;
```

Qué se observó (`e4_interbloqueo_orden_cruzado.txt`):

Sesión	Comando	Salida
A	`UPDATE` Napolitana	`UPDATE 1`
B	`UPDATE` Fugazzeta	`UPDATE 1`
A	`UPDATE` Fugazzeta	`ERROR: 40P01: se ha detectado un deadlock`, con el detalle de los dos procesos que se esperan entre sí
A	`COMMIT`	`ROLLBACK`: la transacción ya estaba abortada
B	`UPDATE` Napolitana	`UPDATE 1`
B	`segundos_desde_begin`	`1.41`
B	`COMMIT`	`COMMIT`

El motor detectó el ciclo, abortó a A y dejó terminar a B.

Explicación de la IA (Kiro, modelo Auto; tal cual en `escenario_4.md`). Identifica el interbloqueo por orden cruzado. Dice que subir el nivel de aislamiento no lo evita y que la solución es que todas las transacciones tomen las filas en el mismo orden. Como alternativas, menciona tomar las dos filas con un solo `SELECT ... FOR UPDATE`, usar `LOCK TABLE` o reintentar ante el `40P01`.

Verificación en el motor.

Variante	Resultado	Evidencia
Mismo orden en las dos sesiones (Napolitana → Fugazzeta)	Sin error: A termina en 1,01 s, B espera y termina en 1,41 s, y las dos confirman	`e4_verificacion_mismo_orden.txt`
Orden cruzado con las dos sesiones en SERIALIZABLE	Mismo `ERROR: 40P01` en A; B confirma	`e4_verificacion_serializable.txt`

Conclusión. La explicación se confirmó: el mismo orden de bloqueo evita el interbloqueo, y SERIALIZABLE no. Kiro cometió dos imprecisiones que no cambian la conclusión:

- Dice que el `UPDATE` toma un `RowExclusiveLock` "sobre la fila". Ese es un bloqueo de tabla, compatible con otros iguales. El conflicto lo produce el bloqueo de fila que también toma el `UPDATE`.
- Dice que un único `SELECT ... FOR UPDATE` con las dos filas las bloquea "en orden interno consistente". PostgreSQL no garantiza ese orden, porque depende de cómo recorra la tabla. Para asegurarlo, hay que ordenar la consulta, por ejemplo con `ORDER BY id`.

Estas dos alternativas no se verificaron en el motor.

Conclusión general

Problema	Lo resuelve	No lo resuelve
Lectura no repetible	REPEATABLE READ, SERIALIZABLE	—
Lectura fantasma	REPEATABLE READ (en PostgreSQL), SERIALIZABLE, `LOCK TABLE ... IN SHARE MODE`	—
Espera por bloqueo	NOWAIT, SKIP LOCKED, `lock_timeout` (evitan esperar)	Subir el nivel: se sigue esperando y además B falla con `40001`
Interbloqueo	Mismo orden de bloqueo	Subir el nivel: el `40P01` sigue apareciendo

La IA acertó en el diagnóstico de los cuatro escenarios, pero no en todos los detalles: en el escenario 3 hizo una afirmación general que el motor desmintió, y en el 4 dio dos datos técnicos imprecisos. Ninguna de esas fallas se notaba leyendo la explicación; aparecieron al repetir el experimento.

DUIA de la Parte 2 (`docs/duia/duia_parte2.md`)

Declaración de uso de IA del laboratorio de concurrencia.

Kiro — explicaciones de los cuatro escenarios

Campo	Completar
Herramienta	Kiro, con el selector de modelo en Auto.
Spec o prompt utilizado	Un chat nuevo por escenario, con el contenido completo de `prompts/explicacion_escenario_1.md` a `4.md`. Esos pedidos los genera `laboratorio_concurrencia.sh reproducir` con los comandos y la salida real de cada sesión, sin título ni pistas sobre el nombre de la anomalía.
Qué generó	Una explicación por escenario: qué pasó y qué nivel de aislamiento o mecanismo de bloqueo lo evitaría.
Qué se aceptó	Las cuatro respuestas se guardaron sin editar en `docs/explicaciones_ia/`.
Qué se modificó o descartó, y por qué	No se modificó ninguna respuesta. Hubo dos errores míos al copiarlas: en el escenario 4 pegué primero la respuesta del 3, así que lo repetí en un chat nuevo; y las respuestas de los escenarios 2 a 4 no quedaron guardadas en el commit `3871360`, por lo que se guardaron en un commit posterior. Los errores de Kiro no se corrigieron en sus respuestas: se documentan en el informe.
Verificación realizada	Repetí cada experimento en el motor con `bash scripts/laboratorio_concurrencia.sh verificar`: REPEATABLE READ y SERIALIZABLE en los escenarios 1 y 2; NOWAIT, SKIP LOCKED, `lock_timeout` y SERIALIZABLE en el 3; y el mismo orden y SERIALIZABLE en el 4. Kiro propuso para el escenario 2 una alternativa que el script no probaba, `LOCK TABLE pedido IN SHARE MODE`, así que la verifiqué con `scripts/verificar_lock_table.sh`. Resultado: Kiro acertó en el diagnóstico de los cuatro escenarios, pero en el 3 afirmó que el nivel de aislamiento no cambia nada, y en SERIALIZABLE la sesión B falló con `40001`. El detalle está en `docs/informe_concurrencia.md`.

Claude (Anthropic)

Campo	Completar
Herramienta	Claude (Anthropic).
Spec o prompt utilizado	La consigna del TP, y después mis evidencias y las respuestas de Kiro.

Campo	Completar
Qué generó	Los fragmentos SQL de las dos sesiones (`food-store/concurrencia/`), el script `laboratorio_concurrencia.sh`, el script extra `verificar_lock_table.sh` y la redacción del informe y de esta DUIA. Antes de dármelo, probó `verificar_lock_table.sh` en una base de prueba con mi esquema y mis datos. También comparó las respuestas de Kiro con las evidencias y señaló las afirmaciones que había que verificar.
Qué se aceptó	Los scripts se usaron como estaban. El informe usa la salida real de mis evidencias.
Qué se modificó o descartó, y por qué	El informe aclara que dos alternativas que mencionó Kiro en el escenario 4 y el comportamiento de REPEATABLE READ en el escenario 3 no se probaron en el motor.
Verificación realizada	Corrí `reproducir`, `verificar` y `verificar_lock_table.sh` en mi PC y controlé los valores clave: 1050 → 1150 en el escenario 1, 2 → 3 en el 2, 1,38 s de espera en el 3 y `40P01` en el 4, y los resultados de cada verificación.

Lo que hice yo

Corrí los escenarios y las verificaciones, pasé cada pedido a Kiro y guardé sus respuestas, revisé la salida de cada evidencia y commiteé cada paso. Tengo que poder explicar en la defensa por qué cada nivel o mecanismo evita, o no evita, cada problema.

Parte 3 · Lectura crítica (`docs/ejercicio_lectura_critica.md`)

Parte 3 de la Semana 2. Los dos scripts dicen «dar de baja registros vencidos» sobre el esquema genérico de cátedra. Antes de corregirlos, se ejecutaron tal como están escritos, dentro de `BEGIN ... ROLLBACK` y sobre un esquema de prueba que se borra al final, para ver en el motor qué hacen realmente:

- Script: `food-store/lectura_critica_demo.sql`, se corre con `bash scripts/lectura_critica.sh`.
- Salida: `evidencias/parte3_lectura_critica.txt`.

La consigna no da las columnas del esquema genérico. Supuse las mínimas para que los scripts tengan sentido:

Tabla	Columnas supuestas
`pelicula`	`id`, `titulo`, `en_cartel` (BOOLEAN)
`funcion`	`id`, `pelicula_id` (FK a `pelicula`, NOT NULL), `activa` (BOOLEAN)
`categoria`	`id`, `nombre`
`producto`	`id`, `nombre`, `categoria_id` (FK a `categoria`, **admite NULL**)

Datos de prueba: tres películas, dos en cartel y una retirada, con dos funciones activas cada una; cuatro categorías, dos con productos y dos sin; y un producto sin categoría.

Script 1

```
-- Generado para: dar de baja las funciones de películas retiradas de cartel
UPDATE funcion
SET activa = FALSE;
```

Qué haría tal como está. No tiene `WHERE`, así que apaga **todas** las funciones de la tabla, de cualquier película. En la prueba: `UPDATE 6`, las seis funciones quedaron inactivas, cuando solo dos eran de la película retirada.

Por qué no cumple lo que dice. El script no mira la película: las funciones de las películas que siguen en cartel también dejan de estar activas. El motor no avisa nada porque la sintaxis es correcta; el único indicio es el número de filas afectadas, y hay que leerlo y compararlo con lo esperado.

Corregido.

```
UPDATE funcion
SET activa = FALSE
WHERE activa
  AND pelicula_id IN (SELECT id FROM pelicula WHERE NOT en_cartel);
```

- La subconsulta elige las películas retiradas. Acá `IN` no tiene el problema del Script 2: la subconsulta devuelve `pelicula.id`, que es clave primaria y no puede ser NULL.
- `WHERE activa` deja afuera las funciones que ya estaban inactivas, así el número de filas afectadas es exactamente lo que se da de baja.
- En la prueba: `UPDATE 2`, solo las dos funciones de la película retirada.

Script 2

```
-- Generado para: limpiar las categorías sin productos asociados
DELETE FROM categoria
WHERE id NOT IN (SELECT categoria_id FROM producto);
```

Qué haría tal como está. Depende de los datos. Si ningún producto tiene `categoria_id` nulo, borra las categorías sin productos, que es lo que se pretende. Pero alcanza con **un** producto sin categoría para que no borre **ninguna**: en la prueba, `DELETE 0`, y las cuatro categorías siguen ahí.

Por qué no cumple lo que dice. `x NOT IN (a, b, NULL)` equivale a `x <> a AND x <> b AND x <> NULL`. Comparar con NULL da desconocido (NULL), así que la condición completa nunca llega a ser verdadera y ninguna fila se borra. La evidencia lo muestra para la categoría 3, que no tiene productos: `3 NOT IN (1, 1, 2, NULL)` da NULL, mientras que sin el NULL, `3 NOT IN (1, 1, 2)` da verdadero.

El script falla en silencio: termina sin error e informa `DELETE 0`, como si no hubiera nada para limpiar. Es el patrón de los casos reales de la consigna: un comando que «funcionó» sin hacer lo que decía.

Corregido.

```
DELETE FROM categoria c
WHERE NOT EXISTS (SELECT 1 FROM producto p WHERE p.categoria_id = c.id);
```

- `NOT EXISTS` pregunta, para cada categoría, si hay algún producto que la referencie. Un producto con `categoria_id` nulo no coincide con ninguna, así que no cambia el resultado.
- En la prueba: `DELETE 2`, se borran las categorías 3 y 4, las dos sin productos.
- También funciona dejar el `NOT IN` y filtrar los nulos en la subconsulta (`WHERE categoria_id IS NOT NULL`). Prefiero `NOT EXISTS` porque no depende de acordarse de ese filtro.

Cómo correría cualquiera de los dos en mi base

Siguiendo `protocolo_seguridad.md`: sobre `foodstore_trabajo` y confirmando antes `SELECT current_database()`; primero un `SELECT COUNT(*)` con el mismo `WHERE` para saber cuántas filas deberían cambiar; después el script dentro de `BEGIN`, comparando el número de filas afectadas con ese conteo. Si coinciden, `COMMIT`; si no, `ROLLBACK`.

Relación con mi proyecto

- En Food Store las categorías y los productos no se borran: se dan de baja con `activo = FALSE` (R7). El equivalente del Script 2 es una baja lógica con el mismo `NOT EXISTS`, mirando solo los productos activos. En la evidencia da `UPDATE 1`: la única categoría activa sin productos activos es Promociones.
- En mi esquema `producto.categoria_id` es `NOT NULL` (R1), así que un `NOT IN` no fallaría por este motivo. Igual uso `NOT EXISTS`: la corrección no depende de que esa columna nunca admita NULL.
- La demostración también repite el error del Script 1, un `UPDATE` sin `WHERE`, sobre mis tablas, dentro de `BEGIN ... ROLLBACK`:
- `UPDATE categoria SET activo = FALSE` falla: la Regla B de la Parte 1 no deja dar de baja una categoría con productos activos, y el motor rechaza la sentencia entera.
- `UPDATE producto SET activo = FALSE` pasa sin ningún error y da de baja los doce productos, porque ninguna regla lo impide.

Las reglas del motor atajan algunos errores, pero no todos. Por eso la lectura previa y la transacción siguen siendo obligatorias.

DUIA de la Parte 3 (`docs/duia/duia_parte3.md`)

Declaración de uso de IA del ejercicio de lectura crítica.

Campo	Completar
Herramienta	Claude (Anthropic). En esta parte no usé OpenCode ni Kiro.
Spec o prompt utilizado	Los dos scripts de la sección 6.3 de la consigna, con el pedido de analizar qué hacen realmente, explicar por qué no cumplen lo que dicen y corregirlos.
Qué generó	El análisis y las correcciones de `docs/ejercicio_lectura_critica.md`, y la demostración `food-store/lectura_critica_demo.sql`, que corre cada script tal como está y corregido sobre un esquema de prueba, dentro de `BEGIN ... ROLLBACK`. La demostración también repite el error del Script 1 sobre mis tablas. Las columnas del esquema genérico son un supuesto, porque la consigna no las da; está aclarado en el documento.
Qué se aceptó	El análisis y las correcciones: para el Script 1, agregar el `WHERE` con la subconsulta de películas retiradas; para el Script 2, reemplazar `NOT IN` por `NOT EXISTS`.
Qué se modificó o descartó, y por qué	Nada.
Verificación realizada	Corrí `bash scripts/lectura_critica.sh` y controlé en `evidencias/parte3_lectura_critica.txt` los siete resultados, en orden. Script 1 tal como está: `UPDATE 6` (todas las funciones); corregido: `UPDATE 2`. Script 2 tal como está: `DELETE 0` (el NULL anula el `NOT IN`); corregido: `DELETE 2`. Sobre mi base: el `UPDATE` sin `WHERE` sobre `categoria` falla por la Regla B; sobre `producto` da `UPDATE 12`, porque ninguna regla lo impide; y la baja lógica con `NOT EXISTS` da `UPDATE 1` (Promociones).

Lo que hice yo

Leí el ejercicio, corrí la demostración y comparé cada resultado con lo que dice el análisis. Tengo que poder explicar en la defensa por qué un `NOT IN` con un NULL en la subconsulta no borra nada.

Anexo A · `food-store/restricciones.sql` (generado con OpenCode)

```
-- Food Store — restricciones.sql (Semana 2, Parte 1)
-- Garantiza por el motor las reglas A, B y C de specs/spec_restricciones.md.
-- Se puede ejecutar más de una vez: los triggers se eliminan con DROP ... IF EXISTS
-- y las funciones se reemplazan con CREATE OR REPLACE.
-- No incluye control transaccional (BEGIN/COMMIT/ROLLBACK): la transacción la abre
-- scripts/aplicar_restricciones.sh. Los BEGIN ... END de las funciones son sintaxis de PL/pgSQL.

-- ============================================================================
-- Regla A — Precio congelado al vender
-- Al insertar una línea, precio_unitario debe coincidir con producto.precio.
-- Una vez registrada la línea, precio_unitario no se puede modificar.
-- No alcanza un CHECK declarativo porque hay que comparar con otra tabla.
-- ============================================================================

CREATE OR REPLACE FUNCTION fn_regla_a_precio_congelado()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    v_precio_actual NUMERIC(10,2);
BEGIN
    IF TG_OP = 'INSERT' THEN
        SELECT precio
          INTO v_precio_actual
          FROM producto
         WHERE id = NEW.producto_id;

        -- Si el producto no existe, la FK de detalle_pedido se encarga del error.
        IF NOT FOUND THEN
            RETURN NEW;
        END IF;

        IF NEW.precio_unitario IS DISTINCT FROM v_precio_actual THEN
            RAISE EXCEPTION
                'Regla A violada: el precio unitario (%) no coincide con el precio actual del producto (%).',
                NEW.precio_unitario, v_precio_actual;
        END IF;
    ELSIF TG_OP = 'UPDATE' THEN
        IF NEW.precio_unitario IS DISTINCT FROM OLD.precio_unitario THEN
            RAISE EXCEPTION
                'Regla A violada: no se puede modificar el precio unitario de una línea ya vendida.';
        END IF;
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_regla_a_precio_congelado ON detalle_pedido;
CREATE TRIGGER trg_regla_a_precio_congelado
    BEFORE INSERT OR UPDATE ON detalle_pedido
    FOR EACH ROW
    EXECUTE FUNCTION fn_regla_a_precio_congelado();

-- ============================================================================
-- Regla B — Baja de una categoría
-- categoria.activo no puede pasar de TRUE a FALSE si la categoría tiene al
-- menos un producto con producto.activo = TRUE.
-- No alcanza un CHECK declarativo porque hay que consultar otra tabla.
-- ============================================================================

CREATE OR REPLACE FUNCTION fn_regla_b_baja_categoria()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF OLD.activo IS TRUE AND NEW.activo IS FALSE THEN
        IF EXISTS (
            SELECT 1
              FROM producto
             WHERE categoria_id = NEW.id
               AND activo IS TRUE
        ) THEN
            RAISE EXCEPTION
                'Regla B violada: no se puede dar de baja la categoría "%" porque tiene productos activos.',
                OLD.nombre;
        END IF;
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_regla_b_baja_categoria ON categoria;
CREATE TRIGGER trg_regla_b_baja_categoria
    BEFORE UPDATE OF activo ON categoria
    FOR EACH ROW
    EXECUTE FUNCTION fn_regla_b_baja_categoria();

-- ============================================================================
```

```
-- Regla C — Fecha de un pedido
-- pedido.fecha no puede ser posterior al momento en que se inserta o modifica
-- la fila. Se usa clock_timestamp() para tomar el momento de la sentencia.
-- ============================================================================

CREATE OR REPLACE FUNCTION fn_regla_c_fecha_pedido()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.fecha > clock_timestamp() THEN
        RAISE EXCEPTION
            'Regla C violada: la fecha del pedido no puede ser posterior al momento actual.';
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_regla_c_fecha_pedido ON pedido;
CREATE TRIGGER trg_regla_c_fecha_pedido
    BEFORE INSERT OR UPDATE ON pedido
    FOR EACH ROW
    EXECUTE FUNCTION fn_regla_c_fecha_pedido();
```

food-store/pruebas_restricciones.sql

```
-- Food Store — pruebas_restricciones.sql (Semana 2, Parte 1)
-- Casos de la tabla de specs/spec_restricciones.md, en el mismo orden.
-- Cada caso es una sentencia suelta: los inválidos muestran el error del motor.
-- No incluye BEGIN, COMMIT ni ROLLBACK: la transacción la abre scripts/aplicar_restricciones.sh.
-- Las filas se buscan por nombre de producto/categoría o por email de cliente, nunca por id.

-- Caso 1 (Regla A) — Resultado esperado: se inserta.
-- Agrega la Napolitana al pedido de Jorge Díaz con el precio actual del producto.
WITH pedido_jorge AS (
    SELECT pe.id
      FROM pedido pe
      JOIN cliente c ON c.id = pe.cliente_id
     WHERE c.email = 'jorge.diaz@mail.com'
     ORDER BY pe.fecha DESC, pe.id DESC
     LIMIT 1
)
INSERT INTO detalle_pedido (pedido_id, producto_id, cantidad, precio_unitario)
SELECT pj.id, pr.id, 1, pr.precio
  FROM pedido_jorge pj
 CROSS JOIN producto pr
 WHERE pr.nombre = 'Napolitana';

-- Caso 2 (Regla A) — Resultado esperado: error.
-- Agrega la Cerveza 1L al pedido de Jorge Díaz con un precio distinto del actual.
WITH pedido_jorge AS (
    SELECT pe.id
      FROM pedido pe
      JOIN cliente c ON c.id = pe.cliente_id
     WHERE c.email = 'jorge.diaz@mail.com'
     ORDER BY pe.fecha DESC, pe.id DESC
     LIMIT 1
)
INSERT INTO detalle_pedido (pedido_id, producto_id, cantidad, precio_unitario)
SELECT pj.id, pr.id, 1, pr.precio + 1
  FROM pedido_jorge pj
 CROSS JOIN producto pr
 WHERE pr.nombre = 'Cerveza 1L';

-- Caso 3 (Regla A) — Resultado esperado: error.
-- Cambia el precio_unitario de una línea ya registrada (Fugazzeta del pedido de Jorge Díaz).
UPDATE detalle_pedido d
   SET precio_unitario = d.precio_unitario + 1
  FROM producto pr,
       pedido pe,
       cliente c
 WHERE d.producto_id = pr.id
   AND d.pedido_id = pe.id
   AND pe.cliente_id = c.id
   AND pr.nombre = 'Fugazzeta'
   AND c.email = 'jorge.diaz@mail.com';

-- Caso 4 (Regla A) — Resultado esperado: se acepta y las líneas vendidas conservan su precio.
-- Cambia el precio de lista de la Muzzarella, que ya tiene ventas.
UPDATE producto
   SET precio = precio + 50
 WHERE nombre = 'Muzzarella';

-- Muestra que las líneas ya vendidas de la Muzzarella conservan su precio original.
SELECT c.email,
       pe.fecha,
       d.precio_unitario,
       pr.precio AS precio_actual
  FROM detalle_pedido d
  JOIN producto pr ON pr.id = d.producto_id
  JOIN pedido pe ON pe.id = d.pedido_id
  JOIN cliente c ON c.id = pe.cliente_id
 WHERE pr.nombre = 'Muzzarella'
```

```
 ORDER BY pe.fecha, c.email;

-- Caso 5 (Regla B) — Resultado esperado: se acepta.
-- Da de baja la categoría Promociones, que no tiene productos.
UPDATE categoria
   SET activo = FALSE
 WHERE nombre = 'Promociones';

-- Caso 6 (Regla B) — Resultado esperado: error.
-- Intenta dar de baja la categoría Pizzas, que tiene productos activos.
UPDATE categoria
   SET activo = FALSE
 WHERE nombre = 'Pizzas';

-- Caso 7 (Regla C) — Resultado esperado: se inserta.
-- Registra un pedido con la fecha de este momento para Carla Ortiz.
INSERT INTO pedido (fecha, forma_pago, cliente_id)
SELECT now(), 'EFECTIVO', id
  FROM cliente
 WHERE email = 'carla.ortiz@mail.com';

-- Caso 8 (Regla C) — Resultado esperado: error.
-- Intenta registrar un pedido con fecha de mañana para Pablo Rivas.
INSERT INTO pedido (fecha, forma_pago, cliente_id)
SELECT now() + INTERVAL '1 day', 'EFECTIVO', id
  FROM cliente
 WHERE email = 'pablo.rivas@mail.com';
```

Anexo B · Explicaciones de la IA, copiadas tal cual

Escenario 1

Herramienta y modelo	Kiro (modelo Auto)
Pedido	`prompts/explicacion_escenario_1.md`, pegado completo en un chat nuevo

> Lo que pasó: Lectura No Repetible (Non-Repeatable Read)
> La sesión A leyó el precio de Muzzarella dos veces dentro de la misma transacción y obtuvo valores distintos: 1050.00 la primera vez y 1150.00 la segunda.
>
> Esto ocurrió porque:
>
> A inicia la transacción y lee el precio → 1050.00
> Mientras A duerme 1.5s, B ejecuta un UPDATE sin BEGIN explícito, por lo que se auto-confirma (autocommit) inmediatamente → el precio queda en 1150.00 en la base de datos
> A vuelve a leer el mismo registro y, como está en READ COMMITTED, cada SELECT ve la última versión confirmada al momento de ejecutarse → ahora ve 1150.00
> El problema es exactamente eso: READ COMMITTED no garantiza que dos lecturas del mismo dato dentro de una transacción devuelvan el mismo resultado. Cada SELECT obtiene un snapshot nuevo del estado confirmado, no uno fijo desde el inicio de la transacción.
>
> Qué anomalía es
> Se llama lectura no repetible (non-repeatable read). El estándar SQL la define como: una transacción lee una fila, otra transacción la modifica y confirma, y la primera transacción vuelve a leer esa fila obteniendo un valor diferente.
>
> Qué evitaría este comportamiento
> REPEATABLE READ (o superior). Con ese nivel, PostgreSQL toma un snapshot del estado de la base al inicio de la transacción y lo mantiene fijo para todos los SELECT de esa transacción, sin importar qué confirmen otras sesiones en el medio.
>
> Si la sesión A hubiera arrancado con:
>
> sql
>
> SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
> la segunda lectura también habría devuelto 1050.00, porque el snapshot fue tomado antes de que B hiciera el commit.
>
> En PostgreSQL, REPEATABLE READ usa MVCC (Multi-Version Concurrency Control): no bloquea la fila ni hace esperar a B; simplemente A sigue viendo la versión de la fila que existía cuando arrancó su transacción. B puede escribir y confirmar libremente, pero A no ve ese cambio hasta que termine su propia transacción.
>
> Resumen de niveles vs. esta anomalía
> Nivel de aislamiento Lectura no repetible
> READ COMMITTED × posible (lo que viste)
> REPEATABLE READ ✓ prevenida
> SERIALIZABLE ✓ prevenida

Escenario 2

Herramienta y modelo	Kiro (modelo Auto)
Pedido	`prompts/explicacion_escenario_2.md`, pegado completo en un chat nuevo

> Qué pasó: Phantom Read (lectura fantasma)
> La sesión A leyó el mismo conjunto de filas dos veces dentro de la misma transacción y obtuvo resultados distintos (2 y luego 3). Esto es exactamente una lectura fantasma: entre las dos lecturas, una transacción externa confirmó una fila nueva que satisface la misma condición del WHERE.
>
> El flujo cronológico:
>
> Tiempo Sesión A Sesión B
> t=0 BEGIN + primer COUNT → 2 —
> t=0.5 pg_sleep(1.5) (durmiendo) INSERT de un nuevo pedido de Ana → COMMIT automático
> t=2 segundo COUNT → 3 —
> t=2+COMMIT —

La sesión B no tenía BEGIN explícito, así que el INSERT se confirmó de inmediato (autocommit). Cuando A hizo su segunda lectura, la fila ya era visible porque READ COMMITTED toma un nuevo snapshot al inicio de cada statement, no al inicio de la transacción.

Por qué READ COMMITTED lo permite
En READ COMMITTED, el snapshot se refresca con cada comando SQL individual. Entonces:

Primer COUNT: snapshot capturado → ve 2 pedidos.
pg_sleep: B inserta y confirma → la fila existe en disco.
Segundo COUNT: nuevo snapshot → ve 3 pedidos.
Esto es comportamiento definido y esperado de ese nivel.

Qué nivel lo evitaría
REPEATABLE READ
En PostgreSQL, REPEATABLE READ toma el snapshot una sola vez al inicio de la transacción y lo mantiene para todos los statements siguientes. Cualquier fila confirmada después de ese punto queda invisible para esa transacción.

Sesión A con REPEATABLE READ:

Primer COUNT → 2 (snapshot fijado aquí).
B inserta y confirma → la fila existe, pero está fuera del snapshot de A.
Segundo COUNT → sigue siendo 2.
No hay fantasma.
Nota: el estándar SQL dice que REPEATABLE READ solo garantiza no tener non-repeatable reads y que los phantoms pueden ocurrir igualmente. PostgreSQL va más allá: su implementación MVCC hace que REPEATABLE READ también elimine los phantoms, porque el snapshot fijo los excluye naturalmente.

SERIALIZABLE
Ofrece el aislamiento más fuerte. Además del snapshot fijo, detecta dependencias entre transacciones y puede abortar alguna si formarían un ciclo de serialización (Serializable Snapshot Isolation). Para este escenario alcanza con REPEATABLE READ, pero SERIALIZABLE también lo resuelve.

Alternativa con bloqueos explícitos (sin cambiar el nivel)
Si se necesita quedar en READ COMMITTED pero evitar que B inserte mientras A está contando, se puede usar LOCK TABLE pedido IN SHARE MODE al inicio de la transacción de A. Eso bloquearía el INSERT de B hasta que A haga COMMIT. Es una solución más agresiva y de menor concurrencia, generalmente no es la preferida frente a simplemente subir el nivel de aislamiento.

Escenario 3

Herramienta y modelo	Kiro (modelo Auto)
Pedido	`prompts/explicacion_escenario_3.md`, pegado completo en un chat nuevo

Qué pasó: Espera por bloqueo (Lock Wait)
El mecanismo: SELECT ... FOR UPDATE
Cuando la Sesión A ejecutó SELECT ... FOR UPDATE sobre la fila de la Coca 1.5L, PostgreSQL adquirió un bloqueo exclusivo de fila (row-level exclusive lock). Ese lock impide que cualquier otra transacción lea esa misma fila con FOR UPDATE o la modifique hasta que A haga COMMIT o ROLLBACK.

La secuencia paso a paso
Tiempo Sesión A Sesión B
t=0 BEGIN + FOR UPDATE → obtiene lock, ve stock=60 —
t=0.5s pg_sleep(2) (duerme) BEGIN + FOR UPDATE → queda bloqueada esperando
t≈2s UPDATE stock = 60-1 + COMMIT → libera lock —
t≈2s — despierta, obtiene lock, lee stock=59
La Sesión B reportó 1.38 segundos desde su BEGIN hasta obtener la fila, que es consistente con haber esperado ~1.5s (arrancó 0.5s después, A durmió 2s; la diferencia es el tiempo de espera real).

Lo clave: B vio stock=59, no 60
Esto es el comportamiento correcto del mecanismo. B no leyó el valor "viejo" de 60; esperó a que A terminara y leyó el valor comprometido actualizado (59). No hay anomalía aquí — el bloqueo funcionó exactamente como debe.

¿Qué nivel de aislamiento o mecanismo evita/controla este comportamiento?
"Evitar" depende de lo que quieras lograr:

Para evitar la espera (no bloquear): usás FOR UPDATE SKIP LOCKED o FOR UPDATE NOWAIT.

> NOWAIT falla de inmediato con un error si la fila está bloqueada.
> SKIP LOCKED omite las filas bloqueadas y sigue adelante (útil para colas de trabajo).
> Para evitar que B vea datos desactualizados sin bloquear: usás el nivel REPEATABLE READ o SERIALIZABLE, que en PostgreSQL usan MVCC. Pero en esos niveles un SELECT sin FOR UPDATE puede leer una snapshot vieja — justo la fuente de anomalías como lecturas no repetibles.
>
> El comportamiento que viste es el diseñado: SELECT ... FOR UPDATE es el mecanismo explícito de serialización pesimista. Garantiza que dos transacciones no modifiquen la misma fila simultáneamente, a costa de que una espere a la otra. Es la herramienta correcta para escenarios como descuento de stock, donde necesitás que la lectura y la escritura sean atómicas respecto a otras transacciones.
>
> Resumen
> Lo que observaste se llama espera por bloqueo (lock wait), no una anomalía. Es el comportamiento esperado de FOR UPDATE bajo cualquier nivel de aislamiento en PostgreSQL. El nivel de aislamiento (READ COMMITTED, que es el default) no cambia esto — los locks de fila son ortogonales al nivel de aislamiento y aplican en todos ellos.

Escenario 4

Herramienta y modelo	Kiro (modelo Auto)
Pedido	`prompts/explicacion_escenario_4.md`, pegado completo en un chat nuevo

> Lo que pasó: interbloqueo (deadlock) por orden cruzado de bloqueos
> Cronología de las dos sesiones
> Las dos transacciones tomaron filas en orden inverso, lo que creó una dependencia circular:
>
> Tiempo Sesión A Sesión B
> t=0 BEGIN —
> t=0.5 — BEGIN
> t≈0 A bloquea Napolitana (UPDATE) —
> t≈0.5 — B bloquea Fugazzeta (UPDATE)
> t≈1 A pide Fugazzeta → espera a B —
> t≈1.5— B pide Napolitana → espera a A
> t≈1.5DEADLOCK detectado → A aborta —
> t≈1.5— B obtiene Napolitana, termina en 1.41 s, COMMIT
> A y B querían las mismas dos filas pero en orden opuesto. Cada una sostenía el bloqueo que la otra necesitaba: ciclo perfecto, deadlock inevitable.
>
> El UPDATE ... SET stock = stock es el detalle clave: no cambia ningún valor, pero PostgreSQL igual adquiere un RowExclusiveLock sobre la fila (un UPDATE sigue siendo un UPDATE). Eso es suficiente para crear el conflicto.
>
> Por qué el nivel de aislamiento no es la solución aquí
> El deadlock ocurrió en READ COMMITTED (el nivel por defecto), pero subiría igualmente a REPEATABLE READ o SERIALIZABLE. El nivel de aislamiento controla qué versión de los datos ves, pero no cambia que dos transacciones que quieren escribir las mismas filas en orden opuesto van a bloquearse mutuamente. SERIALIZABLE en PostgreSQL usa SSI (Serializable Snapshot Isolation) y podría detectar el ciclo, pero igualmente abortaría una de las dos transacciones.
>
> El mecanismo que evita esto: orden consistente de bloqueos
> La solución correcta es un convenio de orden: toda transacción que necesite bloquear más de una fila debe hacerlo siempre en el mismo orden (por ejemplo, por id o por nombre alfabético).
>
> Si ambas sesiones hubieran pedido primero Fugazzeta y luego Napolitana, la segunda transacción simplemente habría esperado en el primer bloqueo sin crear ningún ciclo.
>
> En práctica:
>
> sql
>
> -- Ambas sesiones usan el mismo orden (ej. alfabético por nombre)
> UPDATE producto SET stock = stock WHERE nombre = 'Fugazzeta'; -- primero
> UPDATE producto SET stock = stock WHERE nombre = 'Napolitana'; -- segundo
> Con ese orden, cuando B llega a Fugazzeta y A ya la tiene, B espera. A avanza, toma Napolitana, termina y hace COMMIT. Luego B despierta, toma ambas y también hace COMMIT. Sin ciclo, sin deadlock.
>
> Otras alternativas válidas:
>
> Bloquear ambas filas de una sola vez con SELECT ... FOR UPDATE en una sola sentencia que las incluya a las dos (PostgreSQL las bloquea en orden interno consistente).

> LOCK TABLE si la granularidad de fila no alcanza, aunque es un cañón para este caso.
> Manejo de reintentos en la aplicación ante el error 40P01, que es la práctica defensiva habitual cuando no se puede garantizar el orden.